גיליון תשובות

מספר נבחן:_____

שאלה 1: זיכרון (35 נקודות)

א.

```
template <typename T>
class InorderBinaryTree : public BinaryTree<T>, public Traversable<T> {

protected:
    virtual void copy(const BinaryTree& other) {
        if (other.left == nullptr)
            left = nullptr;
        else
            left = new InorderBinaryTree<T>(*(other.left));
        if (other.right == nullptr)
            right = nullptr;
        else
            right = new InorderBinaryTree<T>(*(other.right));
    }

public:

    InorderBinaryTree(T d) :  BinaryTree(d) {}
    InorderBinaryTree(T d, InorderBinaryTree<T>* l, InorderBinaryTree<T>* r) :
                                    BinaryTree(d,l,r) {}

    virtual void traverse(const Func<T>& f) {
        if (getLeft() != null)
            ((InorderBinaryTree<T>*)getLeft())->traverse(f);
        f.apply(getData());
        if (getRight() != null)
            ((InorderBinaryTree<T>*)getRight())->traverse(f);
    }
};
```

מפתח ניקוד:

- **היעדר מבנה OOP, הפחתה של 1-2 נק׳.**
- **אין השמה לשדות left,right כך שיהיו מטיפוס InOrderBinaryTree, או לחלופין מימוש של copy שמותאם - למעשה אין לעצים אלו את המתודה traverse, הופחתו 3 נק.**
- **לא מומש RO3 בכלל (כלומר,מומשה רק המתודה traverse), הפחתה של 8 נק.**
- **מימוש RO3 בצורה חלקית/שגויה -**
- **מימוש חלקי או שגוי של המתודה traverse (הפחתה של בין 2-3 נק׳)**
- **לא הופחתו נק׳ על היעדר casting במתודה traverse.**
- **שדות ללא צורך וכפולים(שקיימים כבר במחלקת האב), הפחתה של נק׳ אחת.**
- **היעדר הצהרה על הרחבה/מימוש של BinaryTree ו Traversable, הפחתה של נק אחת.**
- **לא הופחתו נק׳ על כך שסטונדטים כתבו בטעות בסינטקס של Java על כך שמרחיבים את המחלקה או מממשים את הממשק(במקרים קיצוניים הופחתה נק׳ אחת)**

ב. שהמעבר על העץ אינו משנה אותו כלל:

- לא נוספים קודקודים חדשים
- לא משתנה הערך בקודקודים הנתונים

מפתח ניקוד:

- **תשובה שלא ציינה <u>מפורשות</u> שהקודקודים אינם משתנים ושלא נוספים קודקודים קיבלה לכל היותר 3 נק'. ניסוח כללי ("האובייקט אינו משתנה") אינו מספק.**

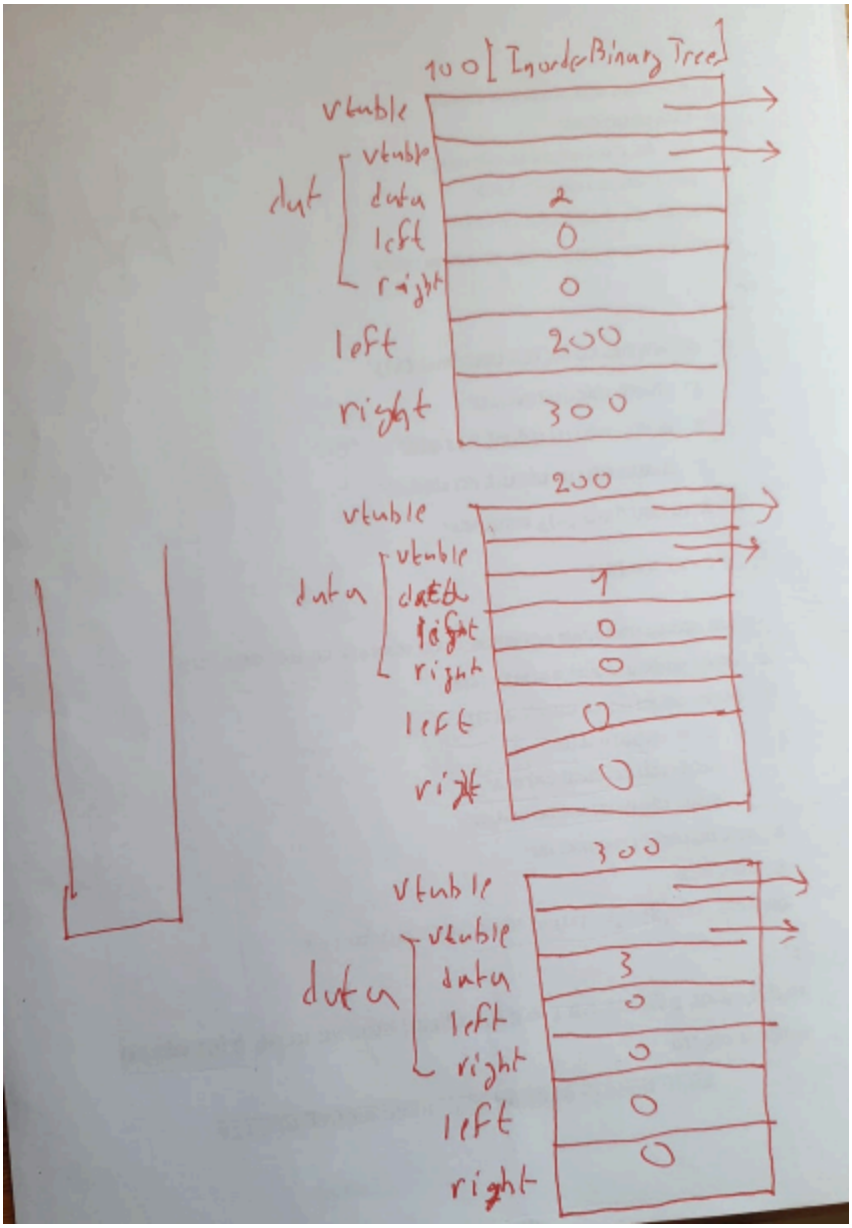
ג.

- התוכנית תתקמפל, כי לא נוספים קודקודים חדשים בהתאם להגדרת traverse כ-const - שמשמעותה שהפונקציה לא מבצעת פעולות השמה על השדות או מתודות מהמחלקה שאינן const.
- אך זה לא מומלץ:
  - ניתן לשנות את הערך בקודקודים, כי הפרמטר item במתודה apply ב'ממשק' Funct אינו const (שימו לב שהגדרת apply כ-const מונעת רק שינוי במחלקה Func), כך שהוא יכול להשתנות.
  - זו לא המשמעות האינטואיטיבית עבור מי שמשתמש ב-traverse (הוא חושב ששום דבר לא משתנה בעץ)

מפתח ניקוד:

- **כדי לקבל ניקוד כלשהו, יש לציין שהתכנית מתקמפלת.**
- **מי שלא טען שההצעה אינה מומלצת, או מי שטען כך ללא נימוק - קיבל לכל היותר 3 נק'.**

ד.



שאלה 2: מקביליות (30 נקודות)

א.

```
public class Barrier {
    private final int threadsNum;
```

```
    private int iDone;

    Barrier(int threadsNum) { this.threadsNum = threadsNum; this.iDone = 0; }

    public int await() throws InterruptedException {
        synchronized (this) {
            iDone++;
            int ret = iDone;
            if (iDone < threadsNum)
                this.wait();
            else {
                iDone = 0;
                this.notifyAll();
            }
            return ret;
        }
    }

    public void reset() {
        synchronized (this) {
            iDone = 0;
            this.notifyAll();
        }
    }

    public int getParties() {
        synchronized (this) {
            return threadsNum - iDone;
        }
    }

    public int getNumberWaiting() {
        synchronized (this) {
            return iDone;
        }
    }
}
```

ב.

```
class Philosopher implements Runnable {

       private Fork left, right;
       private Barrier barrier;

       Philosopher(Fork left, Fork right, Barrier barrier) {
          this.left = left; this.right = right; this.barrier = barrier;
       }

       public void run() {
          while (!Thread.currentThread().isInterrupted()) {
             think();
             barrier.await();
              if (System.identityHashCode(left) <
                   System.identityHashCode(right)) {
                  synchronized(left) {
                    synchronized(right) {
                          eat();
                    }
                  }
              }
              else {
                 synchronized (right) {
                    synchronized  (left) {
                          eat();
                    }
                 }
             }
             barrier.await();
        }
    }

    public static void main(String [] args) {

        Fork[] forks = new Fork[5];
        Barrier barrier = new Barrier(5);
        Philosopher[] philosophers = new Philosopher[5];
        for (int i=0; i<5; i++)
            forks[i] = new Fork();
        for (int i=0; i<5; i++)
            philosophers[i] = new Philosopher(
                                  forks[i % 5],forks[(i + 1) % 5], barrier);
        for (int i=0; i<5; i++)
            new Thread(philosophers[i]).start();
```

```
    }
}
```

ג.

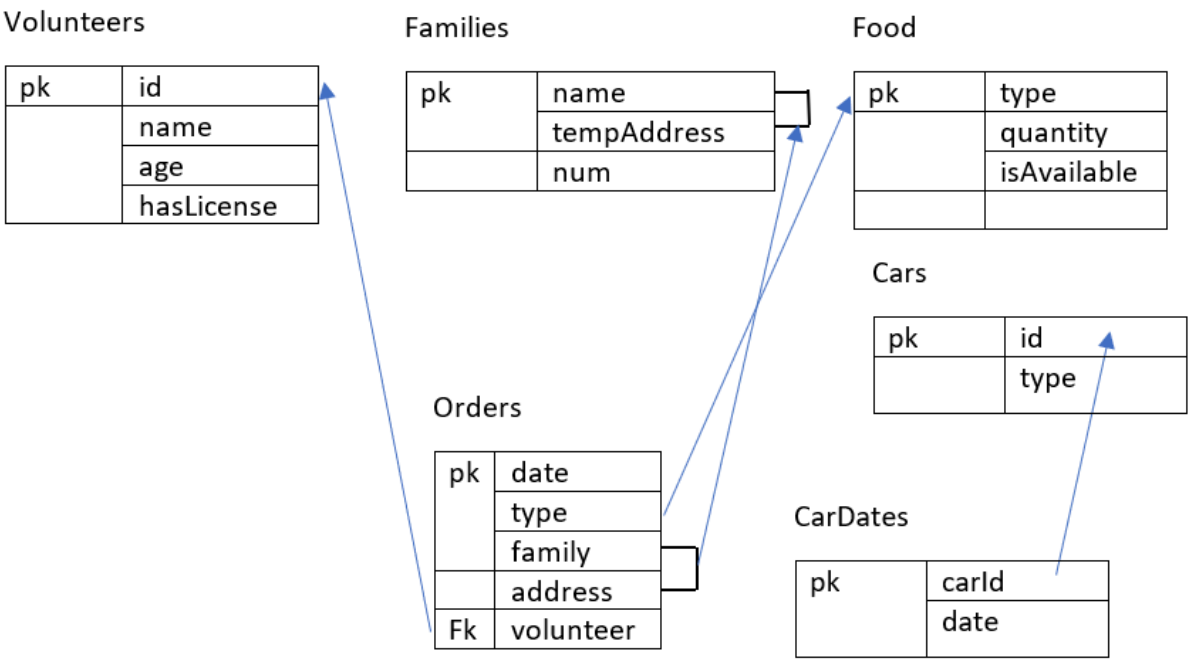
- בקוד המקורי ולאחר העדכון: אין דדלוק כי יש resource ordering בתפיסת המזלגות
- בקוד ולאחר העדכון: אין livelock כי אין חסימה הדדית של הריצה עם תנאי התחלה בקוד.
- כפי שראינו בכיתה, בקוד המקורי קיימת אפשרות תיאורטית של הרעבה בשל השימוש במנגנון סנכרון לא הוגן (synchronized), אך כעת אין סכנה כזאת כי לאחר סיום כל שלב אכילה/חשיבה הת'רדים ממתינים כך שהת'רדים החסומים בהכרח יצליחו לתפוס את המנעול ולאכול/לחשוב. כך גם לגבי wait, כאשר ת'רד מתעורר הוא מיד ממשיך הלאה, בלי קשר לפעולות הת'רדים האחרים.

מפתח ניקוד:

- **כלל הניקוד (10 נק') חולק לשישה חלקים שווים (deadlock, livelock, starvation לפני / לאחר העדכון), ועוגל למספר השלם הקרוב ביותר. למשל, 5 סעיפים נכונים וסעיף אחד שגוי מזכים ב 8 נק'.**
- **על תשובות לא מנומקות ירדו 1-2 נק'.**

שאלה 3: תקשורת ובסיסי נתונים (35 נקודות)

א.



ב.

```
class DBProtocol implements MessagingProtocol {

    private boolean shouldTerminate;
    private DB db;

    public DBProtocol(DB db) { this.shouldTerminate = false; this.db = db; }

    @Override
    public String process(String msg) {
        if (msg.equals("bye"))
           shouldTerminate = true;
        else {
            String res = null;
            try {
                List<String> records = db.executeQuery(msg);
                if (records.size() == 0)
                   res = "No Result";
                else
                   res = records.get(0);
            } catch (SQLException e) {
                   res = e.getMessage();
            }
            teturn res;
        }
    }

    @Override
    public boolean shouldTerminate() {
        return shouldTerminate;
    }
}

class DBServer {
    public static void main(String[] args) {
        int port = IntegerParseInt(args[0]);
        DB db = new DB();
        new Reactor(10, port,
                    () -> return new BBProtocol(db),
                    () -> return new LineMessageEncoderDecoder()).serve();
    }
}
```

ג. על פי הצעת הסטודנט קטגורית ייתכנו כמה תרחישים מקביליים שאינם אפשריים בקוד המקורי של הריאקטור:

- עבודה של שני ת'רדי תקשורת במשימות קריאה/כתיבה על אותו לקוח: נפתר באמצעות הגדרת האקזקיוטור שלהם כ-ActorThreadPool (זהו פתרון מועד א)
- עבודה של שני ת'רדי תקשורת במשימת התחברות עבור לקוחות חדשים שונים: המחלקה ServerSocketChannel אמנם אינה מסונכרנת, אך לא תיתכן מקביליות כזאת שכן האקטור של משימות אלו היא אובייקט הריאקטור המשותף.
- מקביליות בין ת'רד תקשורת המחבר לקוח לבין הת'רד הראשי - כאן יש בעיה:
  - אם ת'רד התקשורת רושם לקוח חדש בסלקטור והת'רד הראשי ישן על הסלקטור (מבצע select) הת'רד הראשי לא ידע על כך, כי הוא הלך לישון על תנאי אחר. כך שאם הלקוח החד שולח הודעות הוא לא יתעורר.
  - ייתכן שת'רד התקשורת בכלל ייתקע בשל הניסיון לשנות את מבנה הנתונים של הסלקטור, כי הת'רד הראשי מבצע כרגע select ולא יתעורר כל עוד אין אירוע אחר.

את שתי הבעיות האלה ראינו בכיתה עבור תרחיש דומה אחר - ב'סוגיית שינוי הרישום בסלקטור'.
הפתרון שנקטנו שם הוא הפתרון עבור מקרה זה:
- רק הת'רד הראשי משנה דברים במבנה הנתונים של הסלקטור
- כאשר ת'רד אחר רוצה לשנות משהו, הוא מוסיף זאת כמשימה לת'רד הראשי ומבצע עליו wakeup.

השינוי הנדרש בקוד הוא במתודה handleAccept:

```
class Reactor {
 ...

 private void handleAccept(ServrSocketChannel serverChan,
                           Selector selector) throws IOException {

     communicationThreads.submit(this, () -> {
        SocketChannel clientChan = serverChan.accept();

        clientChan.configureBlocking(false);
        final NonBlockingConnectionHandler<T> handler = new
             NonBlockingConnectionHandler<>(
                readerFactory.get(),
                protocolFactory.get(),
                clientChan,
                this);
        selectorTasks.add( () -> {
           clientChan.register(selector, SelectionKey.OP_READ, handler);
        });
        selector.wakeup();

     });
   }

   ...
}
```

תשובות על בעיות מסוג "שימוש ב-synchronized גורם לת'רדים לעבור למצב blockecd" וכדומה אינן קבילות, כפי שהוסבר בפיתרון למועד א. גם בקוד הריאקטור יש שימוש ב-synchronized.